Dialog Creator - User Manual

This document describes how to use the Dialog Creator editor window to design dialogs by adding and arranging UI elements on a canvas.

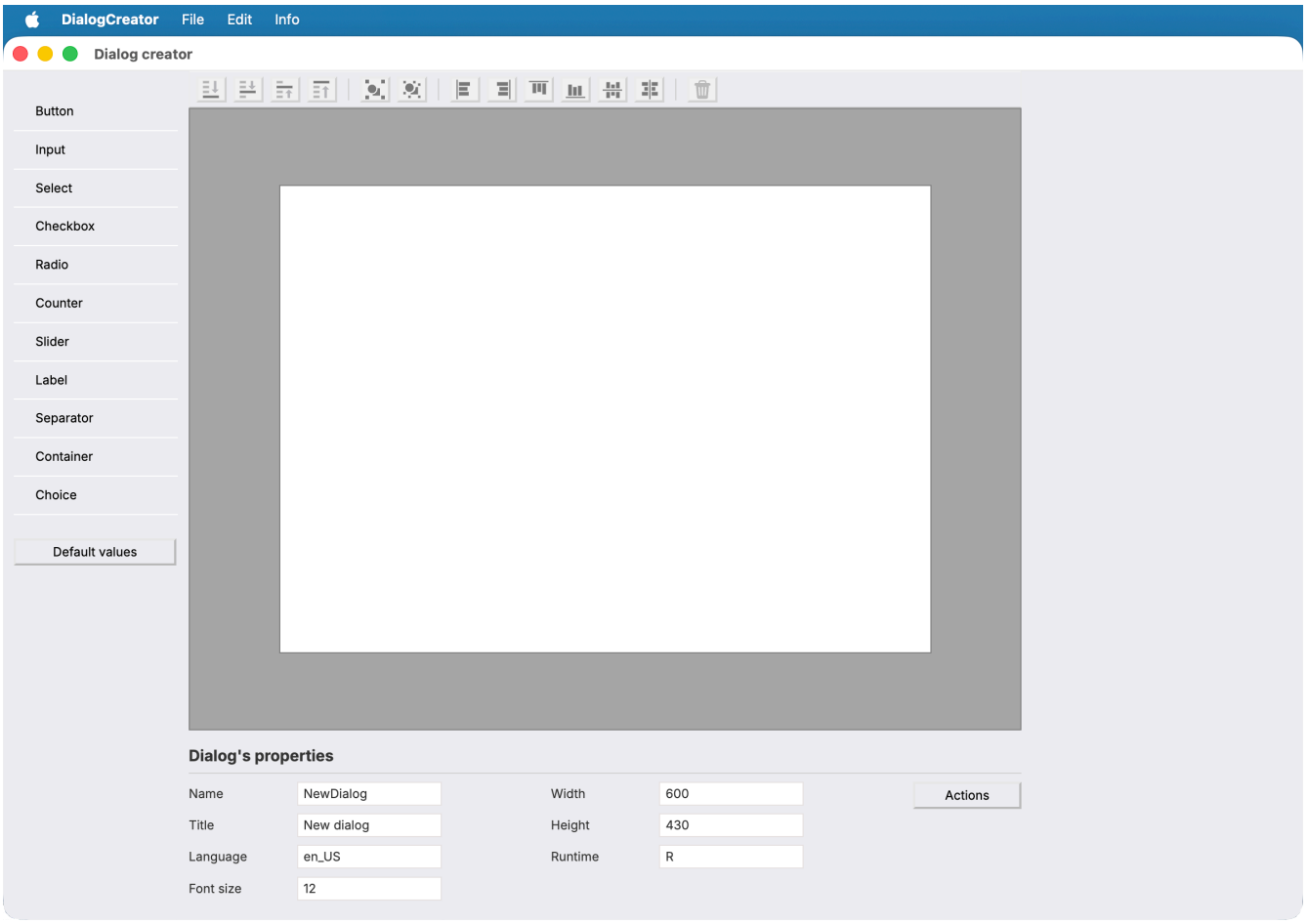
Description

The Dialog Creator is a cross-platform, graphical interface for building dialog layouts by placing various UI elements onto a canvas. It allows users to visually design dialogs by adding, positioning, and configuring elements such as buttons, inputs, labels, checkboxes, and more.

It also allows connecting UI elements, then test custom logic instantly in a live preview.

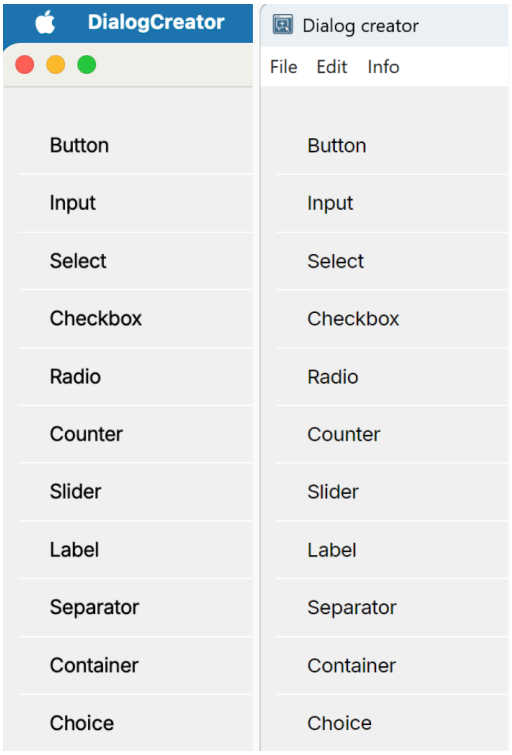
Upon installing and opening the application, the following main window can be seen.

Overview of the interface



This is a fresh instance of the editor window, which can be divided into five main areas.

Elements panel (left)



The image above shows the Elements panel with available UI controls, in both MacOS and Windows styles.

This panel is the catalog of building blocks. It lists all available element types that can be added to a dialog: buttons, labels, inputs, checkboxes, radios, selects, containers, separators, counters, sliders, and more. Items can be clicked to insert a new instance onto the canvas with sensible default properties. Those defaults can be changed for future inserts (for example, a preferred border or font color for a certain element), using the "Default values" button to open a small window where the per-type defaults are located.

Those new defaults are saved with the application and persist across sessions.

Editor toolbar (top of center)



Z-order (stacking) actions for the current selection:

- Send to back
- Send backward
- Bring forward
- Bring to front

Grouping actions:

- Group selected (enabled when 2+ elements are selected)
- Ungroup (enabled when a persistent group is selected)

Alignment (arranging) actions — align elements relative to the first selected (anchor):

- Align left, right, top, bottom
- Align horizontal center (center), align vertical center (middle)

- When aligning multiple targets at once, their relative spacing is preserved (treated as a block) and the block is aligned to the anchor.
- Requires at least two selected elements. Use Shift+click or a lasso selection to select multiple.

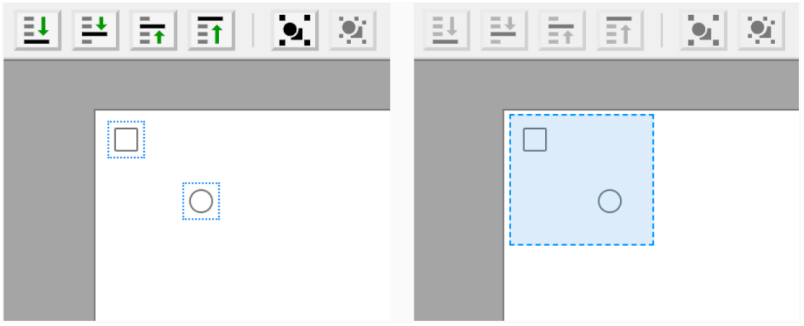
Delete:

- Remove the current selection (single element, multiple elements, or an entire group)
- Also available via Delete/Backspace

All toolbar buttons enable or disable automatically based on what's selected. Selecting elements (to decide for grouping, alignment, or deletion) can be done in two ways: either shift-clicking on each element or using a lasso selection, as per the image above.

Dialog canvas (center)

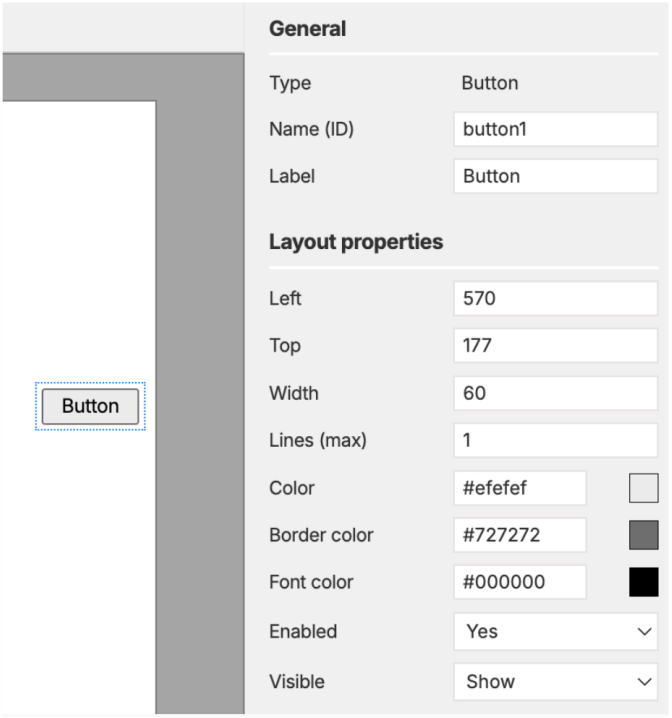
This is the main stage where the dialog is laid out. Newly added elements appear here as movable blocks and show a dotted outline when selected. Clicking an element once will select it and reveal its properties on the right, while clicking on empty space will clear the selection.



Elements can be dragged to reposition them, and their movement is constrained within the canvas with a small padding so items don't slip outside the visible area. Multiple elements can be selected with Shift-click or by drawing a lasso on empty canvas, as per the image above.

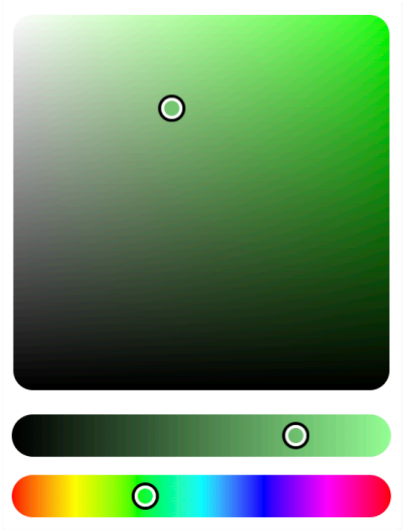
These elements can then be moved together or aligned from the toolbar. Right-clicking an element / group reveals quick actions like Duplicate, Group, or Ungroup.

Properties panel (right)



This panel displays properties for the selected element, in the image above showing a button's properties. Different elements have different sets of properties, so only properties relevant to the selected element type are shown and enabled.

All elements have Left and Top properties to control their position on the canvas. Other properties depend on the element type, such as Label and Color for buttons, Value for inputs and labels, Options for selects, Checked for checkboxes, and so on.



Colors can either be typed as hex codes (e.g., `#FF0000` for red) or selected via a color picker that opens when clicking the color swatch, as in the image above. The color picker dissapears when clicking elsewhere on the dialog, or pressing the ESC key.

Some properties can be overwritten with custom JavaScript code in the Actions window, for example Enable or Visible, using a pre-defined set of API commands, see the details in the API reference.

Dialog properties and Actions (bottom)

The whole dialog has properties of its own, for instance width and height to establish how large it should be, or a name (to be referred to from other dialogs) or a title (shown in the dialog's title bar).

It also has a global font size that affects all elements uniformly, all of which are saved with the dialog and reflected in the live Preview window.

The dialog-level **Language** property sets the source locale of the dialog text (for example `en_US`, `ro_RO`, `de_DE`). This locale is exported with the dialog JSON and used as the base locale for translations in target applications.

The dialog-level **Runtime provider** property carries the target execution provider that the dialog scripts are meant to build commands for, such as `R`, `Python`, or `Julia`. Preview exposes this value to custom JavaScript as `runtimeProvider` so dialogs can branch on provider-specific command syntax when needed.

There is also an Actions button to open the code window for adding custom JavaScript logic for dialog behavior. It can be used to define how the elements interact with each other based on user input: showing/hiding/enabling controls, and updating values programmatically.



Localization in exported JSON

When a dialog is saved, the JSON includes:

- `properties.language`: the source language selected in Dialog properties.
- `i18n.baseLocale`: set from `properties.language`.
- `i18n.locales`: locale dictionaries with translatable strings.

The base-locale dictionary is generated from visible dialog text such as:

- `dialog.title`
- `elements.<id>.label`
- `elements.<id>.value`
- `elements.<id>.items.<index>`

Custom JavaScript messages that are not visible elements, such as validation errors or warning dialogs, are translated automatically by the API functions that display them:

```
addError(c_datasets, 'No dataset selected');
showMessage('No dataset selected', '', 'warning');
```

Message lookup checks the active dialog language first, then the base locale, and finally returns the original text.

Stable message keys can still be preserved in the locale dictionaries. For example, the base locale can store `"messages.noDatasetSelected": "No dataset selected"`, while another locale stores the same key with the translated message. Calling `addError(c_datasets, 'No dataset selected')` will resolve through that stable key.

This data is meant for the target app that imports the dialog. Dialog Creator does not auto-translate the editor UI.

Example:

```
{
  "properties": {
    "name": "NewDialog",
    "title": "Settings",
    "language": "en_US",
    "runtimeProvider": "R"
  },
  "i18n": {
    "baseLocale": "en_US",
    "locales": {
      "en_US": {
        "dialog.title": "Settings",
        "elements.button1.label": "OK"
      },
      "ro_RO": {
        "dialog.title": "Setari",
        "elements.button1.label": "Confirma"
      }
    }
  }
}
```

Description of Elements

Button

The Button element represents a clickable button. It can be styled and configured with different actions to perform when clicked.

Input

The Input element allows users to enter text or data. Internally it behaves like a single-row wrapping text area: long text wraps visually inside the element height, but pressing `Enter` commits the value and removes focus instead of inserting a new line. The `height` property controls how tall the element is, and a vertical scrollbar appears only when the wrapped content exceeds that visible height.

Select (Dropdown)

The Select element provides a dropdown menu for users to choose from a list of options. It can be configured with default selections and multiple selection capabilities.

Checkbox

The Checkbox element allows users to make binary choices (checked or unchecked), generally as a true / false switch when writing the options in the command syntax.

Radio Button

The Radio Button element allows users to select one option from a set of mutually exclusive choices. Radio buttons can be grouped together to form a selection group.

Counter

The Counter element allows users to increment or decrement a numeric value within a specified range. It can be configured with minimum and maximum limits, plus a start value.

Slider

The Slider element provides a graphical interface for selecting a value from a continuous range. It can be customized with different step sizes and value ranges.

Label

The Label element displays static text or information. It can be customized with different font colors, and its size depends on the text content, up to a certain maximum width. If the text exceeds the maximum width, it will be truncated with an ellipsis (`...`). There is a control for how many lines of text are shown before truncation using the Line Clamp property.

Separator

The Separator element is a visual divider used to separate different sections or groups of elements within the dialog.

Container

The Container element is a versatile component that can hold multiple items. It supports single or multi-selection modes and can be populated dynamically via the API. The items in a Container can be selected and deselected by clicking, and multi-selection is supported via Shift+click for range selection. In single-selection mode, clicking the active item clears the selection. Their type can be restricted (e.g., numeric, character, date) so that only items of that type are enabled and selectable.

Its properties panel exposes an **Item type** dropdown alongside the selection mode. It defaults to **Any**, which allows all items to remain interactive. Choose a specific type (Numeric, Factor, Calibrated, Binary, Character, Categorical, or Date) to enforce that only items whose metadata matches the selected type stay selectable. Items with a different type are visually muted, ignore clicks, and are removed from the active selection.

Containers can be populated by code, using `setValue(container, array)` . The array can be either:

- an array of labels, such as: `setValue(container, ["age", "gender"])` to add two plain items, or
- a metadata object containing the item name, type flags, and whether it should be displayed as selected, e.g.:

```
setValue(container, [
  { name: "age", numeric: true, active: true },
  { name: "gender", factor: true, categorical: true }
]);
```

Helpers such as `listColumns()` return precisely such metadata objects; when that output is paired with a Container whose Item type is set, mismatching items will automatically render as disabled.

This element has another useful property called **Item Order**. When activated, the order in which the items are clicked is duly remembered, and `getSelected()` returns selections in that order. For instance in statistics, when creating a cross-tabulation from two categorical variables, it does matter which variable is selected first (on the rows) and which second (on the columns).

Another Container property is **Pin on top**. When enabled, the currently selected items are visually moved to the top of the list in Preview, while all unselected items remain below in their original order. This is useful when working with long lists because the active selection stays visible even after scrolling or filtering the container.

When **Pin on top** is combined with **Item Order**, the pinned rows keep the same click order returned by `getSelected()`. Without **Item Order**, selected rows are still pinned above the others, but ties between selected rows fall back to their original list order.

Choice

The Choice element is a list control built for (ordered) choices. Each item acts like a selectable option, and the list itself can optionally be reordered by drag-and-drop. Because it combines sorting and ordering, it can be used in a wide variety of ways:

- **Multi-select**: when Ordering is disabled, items behave like simple on/off selections from a predefined list.
- **Sorting selector**: choose which fields are active, and (optionally) their direction (ascending/descending).
- **Ranking list**: drag items up and down to establish a priority order (the order of items is the ranking).

Its properties include:

- **Items**: the list of item labels (comma/semicolon separated).
- **Sortable**: if Yes, items can be reordered by dragging.
- **Ordering**: stored values are `increasing`, `decreasing`, or `no`. `Yes, increasing` cycles through `not selected → ascending → descending → not selected`; `Yes, decreasing` cycles through `not selected → descending → ascending → not selected`; `No` cycles through `not selected → selected → not selected` and no arrow is shown.
- **Align**: text alignment for item labels (left/center/right).
- **Colors**: Background/Font/Active background/Active font/Border colors affect the list and the active item styling, just like the Container element.

In custom JS, `getValue(element)` returns the ordered items along with their state (`off`, `asc`, `desc`). `getSelected(element)` returns the active selections; for Ordering-enabled lists, this includes the direction suffix, while for `No` it returns labels only. Programmatic updates via `setValue()` or `setSelected()` may also include direction suffixes such as `income:desc`.

Keyboard shortcuts

Arrange (Z-order) actions:

- Cmd/Ctrl + ↑: Bring forward, moves the element one step forward in stacking order.
- Cmd/Ctrl + Shift + ↑: Bring to front, places the element above all others in the canvas.
- Cmd/Ctrl + ↓: Send backward, moves the element one step backward in stacking order.
- Cmd/Ctrl + Shift + ↓: Send to back, places the element behind all others in the canvas.

These actions are disabled when no element is selected.

Grouping:

- Cmd/Ctrl + G: Group selected
- Cmd/Ctrl + Shift + G: Ungroup selected group

Movement (nudge):

- Arrow keys: Move selected element(s) by 1px
- Shift + Arrow keys: Move selected element(s) by 10px

Global:

- Cmd/Ctrl + A: Select all elements on the canvas (Editor window)

Notes:

- Shortcuts only apply when at least one element is selected and focus is not inside a text field (unless stated otherwise).
- Cmd/Ctrl modifiers are reserved for arrange and grouping actions; nudging uses arrows without Cmd/Ctrl.
- When multiple elements are selected, nudging moves all selected elements together.

Shortcuts cheatsheet

Arrange (Z-order)

Cmd/Ctrl + ↑ Bring forward

Cmd/Ctrl + Shift + ↑ Bring to front

Cmd/Ctrl + ↓ Send backward

Cmd/Ctrl + Shift + ↓ Send to back

Movement (Nudge)

↑ ↓ ← → Move 1px

Shift + ↑ ↓ ← → Move 10px

Delete / Backspace Remove selected

Cmd/Ctrl + A Select all elements

Grouping

Cmd/Ctrl + G Group selected

Cmd/Ctrl + Shift + G Ungroup selected group

Shortcuts apply only when an element is selected and focus is not in an input field.

Working with elements

Add a new element

In the Elements panel (left), click the element to be added. It will be inserted on the dialog canvas with default properties.

Select an element

- Click an element on the canvas to select it.
- A selected element is highlighted with a dotted outline.
- The buttons on the top toolbar are enabled when an element is selected.

Deselect elements

- Click on an empty area of the dialog canvas to clear the selection.
- The buttons on the top toolbar become disabled.

Move an element

- Drag an element to reposition it.
- Movement is constrained within the dialog canvas with a small margin.
- Use Arrow keys to nudge by 1px; hold Shift for 10px steps (when an element is selected and focus is not in an input).

Remove an element

- Press Delete/Backspace (when the focus is not inside a text field) to remove the selected element, or
- Click the Remove button (Trashcan icon) in the top toolbar.

Preview window

- Opens from the File menu (Preview) or using the keyboard shortcut (Cmd/Ctrl + P)
- It renders the dialog with live interactions.
- Disabled elements remain fully visible, only greyed out (no opacity fade). Native inputs/selects retain the exact same size when disabled.
- Pressing ESC closes the Preview window.

Item selections in Preview

- Containers support multi-selection. Clicking an item toggles its selection (active state); in single-selection mode, clicking the active item clears it. A `'change'` event is dispatched on the Container so that handlers can react.
- Select elements are single-choice. Changing the selection dispatches `'change'` like native selects.

Runtime errors in Preview

- When Custom JS misuses the API (e.g., unsupported event, unknown element, invalid select option), a visible error box appears inside the Preview canvas. This helps spot issues without checking the console.
- The error box can be dismissed with ESC.

Custom JS code — quick start

Dialog Creator supports custom JavaScript through a built-in API. For events, helper functions, and the full reference, open the API reference.

This section includes quick-start recipes, container population examples, validation helpers, and practical scripting patterns.



Some dialogs have complex behaviors that require custom JavaScript code. Open the code window with the Actions button at the bottom of the Editor. This code runs at the top level automatically, with a dedicated, provided API.

The stored dialog format is now split on purpose:

- `dialog.json` carries the dialog structure and properties.
- `actions.js` carries the dialog behavior and is the canonical script file for saved dialogs.

Elements can be referred to by their Name (ID) either quoted or not. For example, `getValue(input1)` is the same as `getValue('input1')` .

Notes on missing elements and strict operations:

- For simple getters/setters (getValue/setValue), if a name is not found, reads return `null` (or a safe default) and writes are ignored.
- For event-related or selection operations (on, select), using an unknown element will throw a SyntaxError and show the error overlay in Preview.

Common patterns that can be used:

1. Show the input's value in a label on change

```
onChange(input1, () => {
  const value = "input1: " + getValue(input1);
  setValue(statusLabel, value);
});
```

2. Show or hide a label when a checkbox is toggled

```
onClick(checkbox1, () => show(label1, isChecked(checkbox1)));
```

Which is equivalent to:

```
onClick(checkbox1, () => {
  if (isChecked(checkbox1)) {
    show(label1);
  } else {
    hide(label1);
  }
});
```

3. Show a select value in a label

```
onChange(countrySelect, () => {
  setValue(statusLabel, "Country: " + getValue(countrySelect));
});
```

4. Update text programmatically

```
setValue(statusLabel, "Ready");
```

5. Preserve the old selection-memory behavior as an app-specific external helper

```
onChange(c_datasets, async () => {
  await callExternal('rememberSelectionBy', {
    source: c_datasets,
    dependents: [c_x, c_y]
  });
});
```

This keeps the old idea available as a host-app extension without keeping it in Dialog Creator's core UI API.

6. Make an input act like a search box for a container

```
searchIn(i_search, c_variables);
```

As the user types in the input, the container is filtered live by item name. The match is case-insensitive, partial matches are allowed, and the current search remains active even if the container is repopulated later with `setValue()`.

7. Enable transient `Cmd/Ctrl+F` search for specific containers

```
enableSearch(c_outcome, c_condition);
```

When the pointer is over an enabled registered container, pressing `Cmd/Ctrl+F` opens a transient search box above that container in Preview. Only one transient search box can be open at a time. `Esc` closes it and clears the filter; if it loses focus while empty, it closes automatically.

8. Build the generated command without formatting it by hand

```
updateSyntax(call('using', 'ess', 't.testhv(F3_agea ~ F2_gndr)'));
```

```
using(
  ess,
  t.testhv(F3_agea ~ F2_gndr)
)
```

`call()` puts one argument per line, and `block()` does the same for a group of statements wrapped in braces:

```
updateSyntax(call('using', [
  'ess',
  block([
    '.table <- wtable(B1_polintr, F2_gndr)',
    'print(.table)',
    'chisq.test(.table)'
  ])
]));
```

```
using(
  ess,
  {
    .table <- wtable(B1_polintr, F2_gndr)
    print(.table)
    chisq.test(.table)
  }
)
```

The indentation is worked out from how deeply each piece is nested, so a dialog never needs to write line breaks or count spaces. Arguments that are empty are dropped, which means an optional part of a command can be added unconditionally. See the API reference for the details, including `indent()`.

9. Refer to the selected dataset in the command

```
updateSyntax(call('using', [
  getReference(c_datasets),
  't.testhv(F3_agea ~ F2_gndr)'
]));
```

Use `getSelected(c_datasets)[0]` for the dialog's own logic, and `getReference(c_datasets)` for the text that goes into the command. They are usually the same name, but a target app may have a filter active on that dataset, in which case `getReference()` returns the matching `subset()` call and the command runs on that subset instead:

```
using(
  subset(
    ess,
    F1_cntry == "RO"
  ),
  t.testhv(F3_agea ~ F2_gndr)
)
```

The dialog does not need to know whether a filter is active, and `call()` takes care of indenting the nested `subset()` correctly. In Preview there is no host data environment, so `getReference()` returns the plain dataset name.

Events:

- Buttons and custom checkboxes/radios usually use `'click'`.
- Text inputs can use `'change'` (on blur) or `'input'` (as you type).
- Selects use `'change'`.
- Tip: Prefer the helpers `onClick`, `onChange`, `onInput` for readability.
- Radio groups: pass the group name to `onChange(groupName, handler)` to attach a handler to every radio in that group. Similar to element names, if the group name is a valid identifier (e.g. `radiogroup1`), the quotes may be omitted.

Programmatic events:

- Convenience functions: `triggerChange(name)` and `triggerClick(name)` are shortcuts for triggering 'change' and 'click' events respectively.

Initialization

- The top-level custom code runs after the Preview is ready (elements rendered and listeners attached). Handlers can be directly registered and initial state can be set without extra lifecycle wrappers.
- Event helpers:
  - `onClick(name, fn)`
  - `onChange(name, fn)`
  - `onInput(name, fn)`

```
onClick(button1, () => {
  // do something
});
```

File menu actions

- New: Optionally saves current work, then clears the canvas.
- Load dialog: Load a `.dc.zip` package or an unzipped dialog directory into the editor.
- Save dialog: Save the current dialog as a `.dc.zip` DialogCreator package. The package is a zip file containing `dialog.json` and `actions.js`; unzipping `xyz.dc.zip` produces an editable `xyz/` dialog directory.
- Save as: Save as `.dc.zip`.
- Preview: Open the live preview window.

Multi-selection and grouping

Select multiple elements

- Shift + Click to add or remove elements from the current selection.
- Lasso selection: Click and drag on an empty area of the dialog canvas to draw a selection rectangle. All elements overlapping the rectangle are selected.
  - Hold Shift while lassoing to add to the existing selection instead of replacing it.

Move multiple elements together (ephemeral selection)

- When two or more elements are selected (but not grouped), dragging any selected element will move all selected elements together.
- Arrow key nudging also moves all selected elements together.
- In the Properties panel, the Type field shows "Multiple selection" and only Left and Top are editable; changing these moves the whole selection.

Group selection (persistent group)

- To lock a multi-selection into a single movable unit, click the Group button in the toolbar or press Cmd/Ctrl + G.
- A group container is created around the selected elements. Selecting a child of a group selects the whole group.
- Groups can be moved and nudged like individual elements.

Ungroup

- Select the group container and click Ungroup in the toolbar or press Cmd/Ctrl + Shift + G to return the elements to the top level. The former members remain selected.

Tips and tricks

- Right-click an element or group to access quick actions like Duplicate, Group, or Ungroup.
- Press Enter while editing a property field to commit changes (the editor will blur the field to trigger the update).
- Some numeric fields are constrained (e.g., size within the canvas, line clamp limited to a small maximum). If a value is out of range, the editor will adjust it automatically.
- Element Name (ID) must be unique. If a duplicate is entered, it will be rejected and an error shown.
- Visibility (isVisible) and Enabled (isEnabled) toggles affect how elements render and behave in the editor.

Troubleshooting

- Arrange buttons are disabled
    - Ensure an element is selected. Click an element on the canvas.
- Delete key doesn't remove the element
    - Make sure focus isn't inside a text field. Click on the canvas and try again.
- Property change seems ignored
    - Most properties apply on blur (when the input loses focus). Press Enter or click elsewhere to commit.

Dialog Creator — API Reference

Use this reference when writing custom JavaScript for Dialog Creator. It contains information about window helpers, event utilities, and data APIs available in the preview runtime.

Scripting API — reference

`showMessage(message, detail?, type?)`

- Shows an application message dialog via the host app.
- message is the visible header; detail is the body text; type (optional) controls icon: 'info' | 'warning' | 'error' | 'question'.
- The message and detail strings are translated automatically when the active dialog locale has matching entries.
- Examples:
- `showMessage('Hello')`
- `showMessage('Low disk space', 'Please free up 1GB', 'warning')`
- `showMessage('Save failed', 'The dialog failed to save your changes.', 'error')`

`getValue(element)`

- Get the element's value/text.
- Input/Label/Select/Counter return their current value; Checkbox/Radio return their current boolean state.
- Choice returns an array of objects shaped like `{ text, state }`, in the current visible order. `state` is one of `off`, `asc`, or `desc`.
- Returns `null` if the element doesn't exist.

`setValue(element, value)`

- Set the value/text.
- Input/Label: set string; Counter: set number within its min/max; Select: set selected option by value; Checkbox/Radio: set boolean state.
- For Container, pass an array of strings or objects shaped like `{ name, numeric, active }`, where type flags are boolean fields.
- For Container with **Pin on top** enabled, selected rows are re-rendered above unselected rows in Preview after the value update. If **Item Order** is also enabled, pinned rows follow the stored selection order.
- For Choice, pass either an array of strings like `['var1:asc', 'var2:desc']`, plain strings like `['var1', 'var2']`, or objects shaped like `{ text, state }`.
- For Choice with Ordering set to `no`, any active state is normalized to a simple selected state. For Ordering set to `increasing` or `decreasing`, both `asc` and `desc` are preserved.
- No-op if the element doesn't exist. Does not dispatch events automatically.

`isChecked(element)`

- For Checkbox/Radio, returns the live checked/selected state as a boolean.

`check(...elements)` / `uncheck(...elements)`

- Convenience methods for Checkbox and Radio elements to set on/off.
- For Radio, `check(element)` also unselects other radios in the same group.
- Passing multiple radios from the same group in a single `check(...)` call throws an error.

- These do not dispatch events by themselves; for the handlers to run, use `triggerChange()` or `triggerClick()`.

`getSelected(element)`

- Read the current selection(s) as an array of values.
- For Select, returns a single-item array (or empty array if nothing selected).
- For Container, returns labels of all selected rows. If Item Order is enabled, the array is returned in the click order.
- For Container with Pin on top enabled, this changes only the visible row order in Preview; the returned values still follow the selection-order rule above.
- For Choice, returns the active items. With Ordering set to `increasing` or `decreasing`, each selected value includes the direction suffix, for example `['var1:asc', 'var2:desc']`. With Ordering set to `no`, it returns only the selected labels.

`isVisible(element)` : boolean

- Returns whether the element is currently visible (display not set to 'none').

`isHidden(element)` : boolean

- Logical complement of `isVisible(element)`.

`isEnabled(element)` : boolean

- Returns whether the element is currently enabled (not marked as disabled).

`isDisabled(element)` : boolean

- Logical complement of `isEnabled(element)`.

`show(element, on = true)`

- Show or hide by boolean. Use `show(element, true)` to show; `show(element, false)` to hide.

`hide(element, on = true)`

- Convenience inverse of show: `hide(element)` hides, `hide(element, false)` shows. Internally calls `show(element, !on)`.

`enable(element, on = true)`

- Enable or disable by boolean. Use `enable(element, true)` to enable; `enable(element, false)` to disable.

`disable(element, on = true)`

- Convenience inverse of enable: `disable(element)` disables, `disable(element, false)` enables. Internally calls `enable(element, !on)`.

`onClick(element, handler)`

- Shortcut for `on(element, 'click', handler)`.

`onChange(element, handler)`

- Shortcut for `on(element, 'change', handler)`.

`onInput(element, handler)`

- Shortcut for `on(element, 'input', handler)`.

`searchIn(input, ...containers)`

- Makes an Input behave like a live search box for one or more Container elements.
- Filtering is case-insensitive and matches item names by partial text.
- Matching rows stay visible; non-matching rows are hidden.
- Search filtering does not clear the current selection automatically.
- The current search remains active if the target container is later repopulated with `setValue()`.
- Throws a SyntaxError if the first argument is not an Input, if a target does not exist, or if a target is not a Container.

`enableSearch(...containers)`

- Enables transient `Cmd/Ctrl+F` search for the specified Container elements in Preview.
- The hovered enabled container becomes the search target when the shortcut is pressed.
- Only one transient search box can be open at a time.
- `Esc` closes the current transient search and clears its filter.
- If the transient search input loses focus while empty, it closes automatically.
- Throws a SyntaxError if no targets are provided, if a target does not exist, or if a target is not a Container.

`setSelected(element, value)`

- Programmatically set selection.
- For Select elements: sets the selected option by value (single-choice).
- For Container elements: accepts a string or array of strings and replaces the current selection with exactly those labels.
- For Container with Pin on top enabled, the new selection is immediately moved to the top of the visible list in Preview. If Item Order is enabled, the pinned rows follow that stored selection order.
- For Choice elements: accepts a string or array of strings. Values may include a direction suffix such as `income:desc`. For Ordering set to `increasing` or `decreasing`, the first click direction follows the configured mode, but both directions remain available when selected programmatically or by subsequent user clicks.
- Does not dispatch a `change` event automatically. For the handlers to run, call `triggerChange(element)` after changing selection.
- Throws a SyntaxError if the element doesn't exist, the control is missing, the option/row is not found, or the element type doesn't support selection.

`clearContent(...elements)`

- Clears the content/value of supported elements.
- Supported: Input (clears the text), Container (removes all rows), Choice (removes all items).
- Throws an error if used on unsupported types.

`setLabel(element, label)`

- Set the visible label text of a Button element.
- Throws a SyntaxError if the element doesn't exist or isn't a Button.

`changeValue(element, oldValue, newValue)`

- Rename a specific item within a Container from `oldValue` to `newValue`.

- If the item is currently selected, the container's selection mirror is updated accordingly.
- No event is dispatched automatically; call `triggerChange(element)` for the change handlers to run.
- Throws a SyntaxError if the element doesn't exist or isn't a Container.

`updateSyntax(command)`

- Updates the Syntax Panel with the provided command string. The panel remains open alongside the Preview window and mirrors its width; closing either window also closes the other.
- Content is rendered with preserved whitespace/line breaks in a monospace font.
- If the floating Syntax Panel cannot be created, a fallback inline panel appears immediately below the Preview canvas inside the Preview window.
- Example:

```
const cmd = buildCommand();
updateSyntax(cmd);
```

`resetDialog()`

- Reset the Preview dialog back to its initial state (reloads the dialog snapshot, restoring default values and selections).

`closeDialog()`

- Requests that the dialog be closed.
- In Dialog Creator Preview, this closes only the Preview window.
- In a target app, this is intended as a semantic signal that the dialog should be closed by the host.

`run(command)`

- Sends the specified command to the backend for execution (for instance to run the R code, if running on top of R).

`callExternal(name, parameters?)`

- Calls an app-specific external function.
- Use this when a dialog needs behavior that is intentionally outside Dialog Creator's documented UI API.
- App-specific helpers such as remembering dependent selections should live here rather than in the shared scripting API.
- In a consuming app, `name` should map to a host-owned implementation registry rather than to a dialog-local helper file.
- `name` is the external function identifier; `parameters` is any serializable object or value the target app expects.
- Returns a Promise.
- In Dialog Creator Preview, this is a no-op and resolves silently so dialogs that use app-specific extensions remain valid here.
- Example:

```
onChange(c_y, async () => {
  await callExternal('xyplot', {
    dataset: getSelected(c_datasets)[0] || '',
    xVariableName: getSelected(c_x)[0] || '',
    yVariableName: getSelected(c_y)[0] || '',
    showCaseLabels: isChecked(cases),
    jitter: isChecked(jitter)
```

```
  });
});
```

- Migration example: if a target app wants to preserve the old `rememberSelectionBy()` convenience, it can expose the same behavior through `callExternal()`:

```
await callExternal('rememberSelectionBy', {
  source: c_datasets,
  dependents: [c_x, c_y]
});
```

- That keeps the behavior available without making it part of Dialog Creator's shared UI API.

Validation and highlight helpers

`addError(element, message)`

- Show a tooltip-like validation message attached to the element and apply a visual highlight (glow). Multiple distinct messages on the same element are de-duplicated and the first one is shown. The highlight is removed automatically when all messages are cleared.
- The message is translated automatically when the active dialog locale has a matching entry.

`clearError(element, message?)` / `clearError(...elements)`

- Clear a previously added validation message. If `message` is provided, only that message is removed; otherwise, all messages for the element are cleared.
- When `message` is provided, it is translated with the same lookup as `addError()` so callers can pass the same source text to add and clear an error.

Object and data helpers

`bindObjects(request)`

- Connects a dialog's object selector to the host application's current data environment.
- Use this when a dialog lets the user choose a dataset, table, or other runtime object and should stay in step with the host application.
- The host owns the object list, refresh behavior, selection memory, and dependent variable or field lists. The dialog should react to selections with `onChange()`, but it should not also fill the same selector manually with `setValue(..., listObjects(...))`.
- `request.datasets` is the container that lists the available datasets or table-like objects.
- `request.variables` is optional. It names one or more containers that should list fields from the selected dataset. It can be a single container, an array of containers, or a role map such as `{ rows: c_rows, columns: c_columns }`.
- `request.dialog` is optional. It identifies the dialog so a host application can apply dialog-specific defaults or policies without exposing product-specific function names in the dialog script.
- Returns a binding handle **synchronously**. Declaring a binding is immediate; only the data refresh is asynchronous, so the call is not awaited.
- The handle exposes `refresh()`, which re-reads the object list and any dependent field containers and returns a Promise. Keep the handle and call it when the dialog resets.
- `refresh()` is safe to call as often as needed, and `bindObjects({ ... }).refresh()` is a valid shorthand when the handle is not kept.
- In Dialog Creator Preview, `bindObjects()` uses the built-in example datasets so the dialog can be tested without a target application.

- In a consuming application, `bindObjects()` is a standard API method that the host must implement. Do not write `callExternal('bindObjects', ...)` as a substitute; `callExternal()` is for app-specific extensions outside the shared API.
- Example:

```
const objectBinding = bindObjects({
  dialog: 'recode',
  datasets: c_datasets,
  variables: c_variables
});

onClick(b_reset, () => {
  resetDialog();
  objectBinding.refresh();
});

onChange(c_datasets, () => {
  clearError(c_datasets);
  selected_dataset = getSelected(c_datasets)[0] || '<dataset>';
  selected_variable = '<variable>';
  updateSyntax(buildCommand());
});
```

`listObjects(type)`

- Returns an array of object names available in the backend environment for the requested `type`.
- Use `listObjects('datasets')` only when the dialog deliberately manages a manual dataset list itself.
- For normal hosted dataset selectors, prefer `bindObjects()` so the host application owns refreshes, selection state, and dependent variable lists.
- Other strings can be used for backend-specific object groups such as arrays, lists, or custom classes.

`listColumns(dataset)`

- Returns an array of column metadata objects available in the specified dataset. Each object has a `name` field and boolean type flags such as `numeric`, `factor`, `calibrated`, `binary`, `character`, `categorical`, and `date`.

`getReference(element)`

- Returns the object selected in `element`, written the way the generated command should refer to it.
- Use it wherever a command needs to name the selected dataset. `getSelected(element)[0]` gives the plain name for the dialog's own logic; `getReference(element)` gives the text to put in the command.
- Returns an empty string when nothing is selected.
- It is synchronous, so a command builder stays an ordinary function.

Most of the time the two are the same:

```
getSelected(c_datasets)[0];   // "ess"
getReference(c_datasets);     // "ess"
```

They differ when the host application has narrowed the object. A target app may let the user keep a filter on a dataset, in which case the command has to run on that subset rather than on the whole dataset. The host reports that through `getReference()`, and the dialog does not have to know it happened:

```
getSelected(c_datasets)[0];   // still "ess"

getReference(c_datasets);
// subset(
//   ess,
//   F1_cntry == "RO"
// )
```

Because `call()` re-indents whatever it is given, that multi-line reference drops into the command without any extra handling:

```
const buildCommand = () => {
  return call('using', [
    getReference(c_datasets),
    't.testhv(' + selected_testvar + ' ~ ' + selected_groupvar + ')'
  ]);
};
```

```
using(
  subset(
    ess,
    F1_cntry == "RO"
  ),
  t.testhv(F3_agea ~ F2_gndr)
)
```

and with no filter active the same code produces:

```
using(
  ess,
  t.testhv(F3_agea ~ F2_gndr)
)
```

- In Dialog Creator Preview there is no host data environment, so `getReference()` always returns the plain selected name and the first form is what appears in the syntax panel.

Syntax building helpers

These describe the *shape* of a generated command and work out the indentation from the actual nesting depth. A dialog script should not contain hand-written line breaks or runs of spaces. They only arrange whitespace, so they behave the same whatever the dialog's Runtime provider is.

`call(name, ...args)`

- Builds a call rendered with one argument per line.
- Arguments can be passed individually or as an array: `call('using', a, b)` and `call('using', [a, b])` produce the same result. Arrays are flattened, so a script can collect arguments in a list and push the optional ones conditionally.
- Empty strings, `null`, and `undefined` arguments are dropped, so an optional argument can be pushed without a surrounding `if`.
- Any argument that spans several lines is re-indented to its real depth. Nested `call()` and `block()` results therefore place themselves correctly.
- With no arguments it returns `name()`.

```
updateSyntax(call('using', 'ess', 't.testhv(F3_agea ~ F2_gndr)'));
```

```
using(
  ess,
  t.testhv(F3_agea ~ F2_gndr)
)
```

`block(...statements)`

- Builds a brace-delimited block with one statement per line, indented one level.
- Handles arguments like `call()`: arrays are flattened and empty parts are dropped.
- With no statements it returns `{}`.

```
updateSyntax(call('using', [
  selected_dataset,
  block([
    '.table <- wtable(' + rows + ', ' + cols + ')',
    'print(.table)',
    isChecked(cb_chisq) ? 'chisq.test(.table)' : '',
    isChecked(cb_fischer) ? 'fischer.exact(.table)' : ''
  ])
]));
```

```
using(
  ess,
  {
    .table <- wtable(B1_polintr, F2_gndr)
    print(.table)
    chisq.test(.table)
    fischer.exact(.table)
  }
)
```

The two unchecked options above contribute empty strings, and those are dropped rather than leaving blank lines behind.

Because a nested call is re-indented automatically, a dataset reference that is itself a call needs no special handling:

```
call('using', [
  call('subset', ['ess', 'F2_gndr == 1']),
  't.testhv(F3_agea ~ F2_gndr)'
]);
```

```
using(
  subset(
    ess,
    F2_gndr == 1
  ),
  t.testhv(F3_agea ~ F2_gndr)
)
```

`indent(text, levels = 1)`

- Shifts every line of `text` right by `levels` levels of two spaces. Blank lines are left alone, so no trailing whitespace is introduced.
- `call()` and `block()` already indent whatever they are given, so `indent()` is only needed for text assembled by hand.

Element-specific details

- Input
  - Read: `getValue(element)` : returns a string
  - Write: `setValue(element, 'hello')`
  - Events: 'change' (on blur) or 'input' (as you type)
- Label
  - Read: `getValue(element)` : returns a string
  - Write: `setValue(element, 'New text')`
- Select
  - Read: `getValue(element)` : returns a string
  - Write: `setValue(element, 'RO')`
  - Event: 'change'
- Checkbox
  - Read state: `isChecked(element)` : returns a boolean
  - Write state: `check(element)` and `uncheck(element)`
  - Event: 'click'
- Radio
  - Read state: `isChecked(element)` : returns a boolean
  - Write state: `check(element)` and `uncheck(element)`
  - Event: 'click'
- Counter
  - Set value within its min/max: `setValue(element, 7)`
  - Read current number: `getValue(element)`
- Button
  - Pressed feedback is built-in in Preview; the handler can trigger other UI changes.
  - Event: 'click'
- Slider
  - Dragging is supported in Preview, and sliders react to changes.
- Container
  - Read current items: `getValue(element)`
  - Read active items: `getSelected(element)`
  - Replace the item list: `setValue(element, [...])`
  - Replace the current selection: `setSelected(element, [...])`

    - If Item Order is enabled, `getSelected()` returns selected values in click order rather than visible row order.
    - If Pin on top is enabled, selected rows move to the top of the visible list in Preview while unselected rows keep their original relative order.
    - When Pin on top and Item Order are both enabled, the pinned block follows the same selection order returned by `getSelected()` .
- Choice
    - Read current ordered state: `getValue(element)` returns `[{ text, state }, ...]`
    - Read active items only: `getSelected(element)`
    - Replace current ordering/selection: `setValue(element, ['income:desc', 'age:asc'])`
    - Replace selected items only: `setSelected(element, ['income:desc', 'age:asc'])`
    - Ordering modes:
        - `no` : cycles `off → selected → off` , and `getSelected()` returns labels only.
        - `increasing` : cycles `off → asc → desc → off` ; the first selection starts with ascending.
        - `decreasing` : cycles `off → desc → asc → off` ; the first selection starts with descending.
    - If Sortable is enabled, the array returned by `getValue()` follows the visible drag-and-drop order.

Practical patterns

- Conditional show a panel when a checkbox is checked:

```
onClick(myCheckbox, () => {
  show(myPanel, isChecked(myCheckbox));
  // or: hide(myPanel, isUnchecked(myCheckbox))
});
```

- Mirror an input's text to a label on change:

```
onChange(myInput, () => setValue(myLabel, getValue(myInput)));
```

- Select a value in a Select (no auto-dispatch), then notify listeners:

```
setSelected(countrySelect, "RO");
triggerChange(countrySelect);
```

- Conditional enable/disable situations:

```
onClick(lockCheckbox, () => {
  disable(saveBtn, isChecked(lockCheckbox)); // disable when locked
  // Equivalent forms:
  // enable(saveBtn, isUnchecked(lockCheckbox));

  // Unconditional forms:
  // enable(saveBtn);            // just enable
  // disable(saveBtn);           // just disable
});
```

- Replace a Container's selection (multi-select) and notify listeners:

```
setSelected(variablesContainer, ["Sepal.Width"]);
triggerChange(variablesContainer);
```

- Delegate dataset-specific selection memory to the host app when needed:

```
onChange(c_datasets, async () => {
  await callExternal('rememberSelectionBy', {
    source: c_datasets,
    dependents: [c_x, c_y]
  });
});
```

- Make an input act as a search box for a container:

```
searchIn(i_search, c_variables);
```

- Enable transient `Cmd/Ctrl+F` search for specific containers:

```
enableSearch(c_outcome, c_condition);
```

- Add or remove items in a Container:

```
addValue(variablesContainer, "Sepal.Length");
clearValue(variablesContainer, "Sepal.Width");
```

- Update a Button label and rename a Container item:

```
setLabel(runBtn, "Run Analysis");
changeValue(variablesContainer, "Sepal.Length", "Sepal Len");
```

Notes

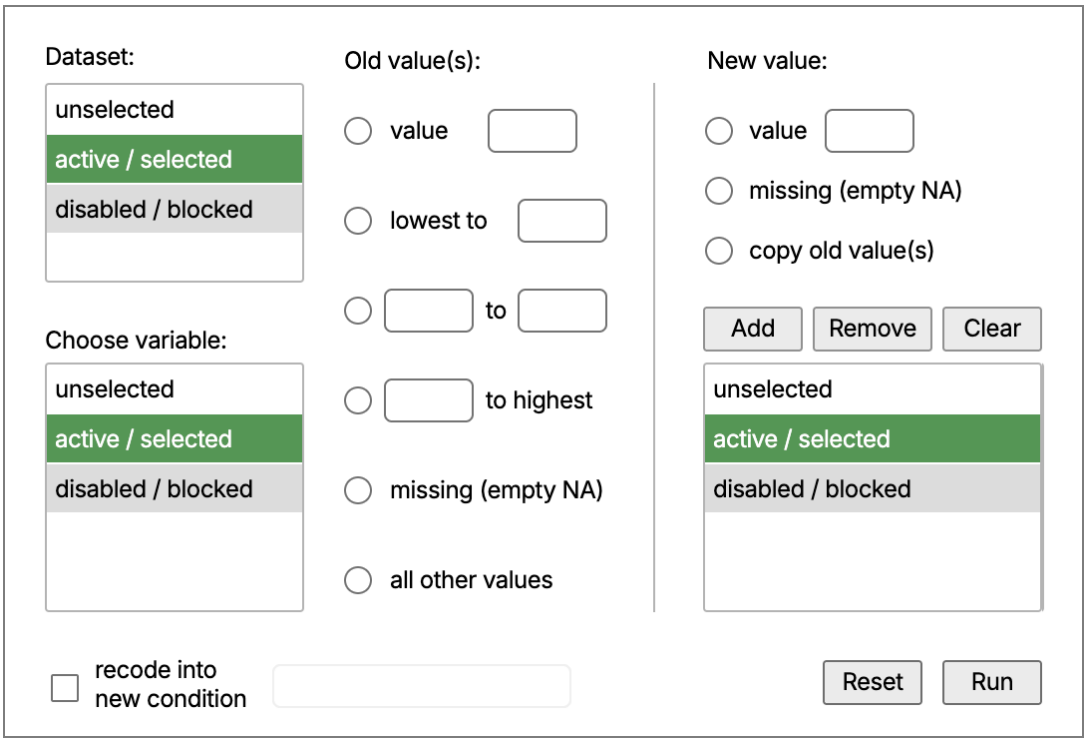
- Programmatic state changes (e.g., `check`, `setValue`) do not automatically dispatch events. Use `triggerChange()` or `triggerClick()` if the dialog should behave as if the user had interacted with the element.
- The selection command (`setSelected`) also does not auto-dispatch, but it can be paired with `triggerChange(element)` to trigger a change event.
- Validation helpers (`addError`, `clearError`) are purely visual aids in Preview; they do not block execution or change element values.

Case study: recode variables dialog

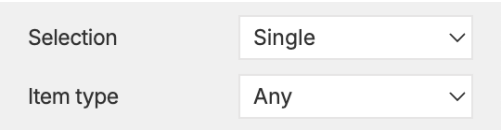
The following section exemplifies, using a step by step approach, how to build a dialog that allows users to recode a variable in the R language. It shows how to construct the dialog in the editor area, and what actions are needed in the scripting area to make it functional and responsive.

The dialog metadata also includes a **Runtime provider** field. It defaults to `R`, but the same dialog structure can target `Python`, `Julia`, or any other provider if the custom JavaScript builds the corresponding command syntax from `runtimeProvider`.

Similar to any other recoding dialog, the user needs to first select a dataset from a list, then choose a variable from that dataset to recode, and finally specify the recoding rules.



The design window contains three Container elements: the top left for datasets and the bottom left for variable, and the right one for the recoding rules. Out of all container properties, the image below highlights the important ones for the dataset container, namely the selection type (single/multiple) and the item type (used for filtering variables). In this particular container, a single dataset can be selected, and the item type is left to the default 'Any' but it could have been set to 'Character', as dataset names are strings.

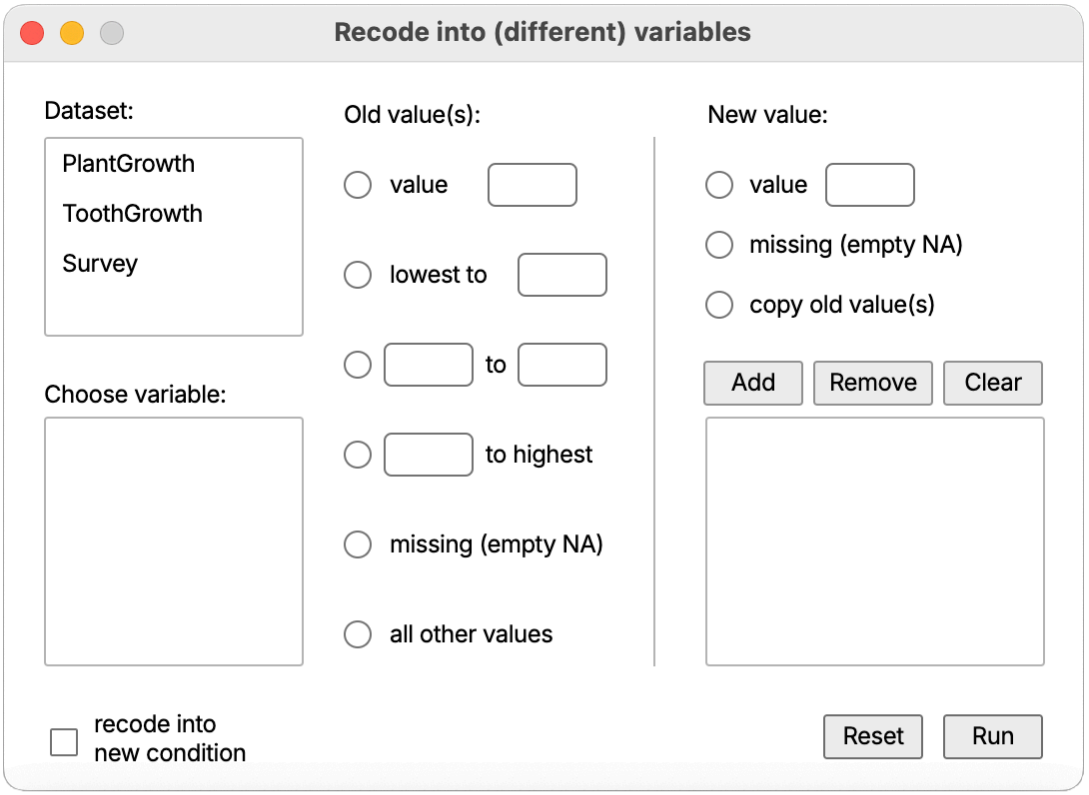


The variables container is also set to single selection, but its item type is set to 'Numeric' to restrict only numeric variables to be selected for recoding. The recoding rules container is set to multi-selection and its item type is also left to 'Any' since it will contain ad-hoc user-defined rules.

The design window also contains two sets of radio buttons, six to specify the old values in the left side of the vertical separator, and three radio buttons on the right side to specify the new values. Above the rules container, there are three buttons to add rules, remove selected rules, or completely clear the entire rules container.

On the bottom part of the dialog, there is a left checkbox to indicate whether the recoding should be done in place (overwriting the original variable) or to a new variable, and a text input to specify the name of the new variable. This input is barely visible in the image because its property 'Visible' is set to 'Hide' by default, and it will be shown only when the checkbox is checked (indicating that a new variable is to be created).

On the bottom right side, there is another (main) button to execute the recoding operation. This button will be responsible with sending / running the final command into R.

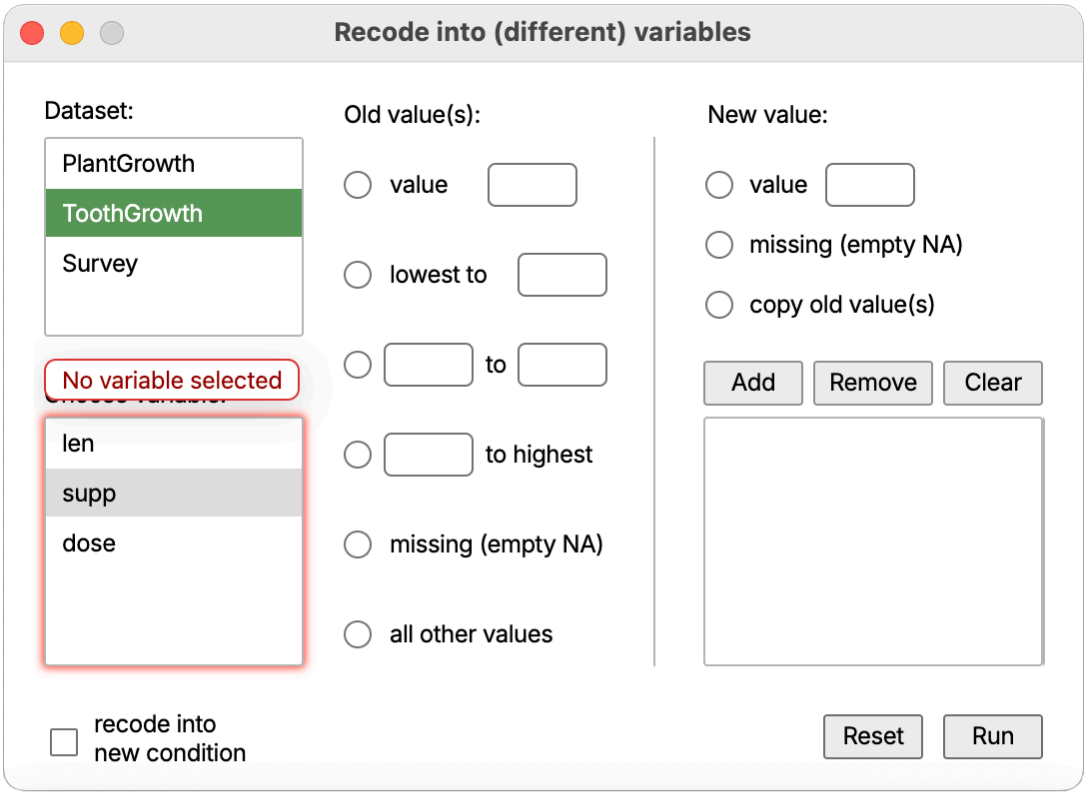


This image above is the preview window of the dialog, showing how it will look like when executed. The datasets container is populated with the list of available datasets, for the time being a simulation of two datasets from the R environment: 'PlantGrowth' and 'ToothGrowth'. Below the window is the syntax panel, which will display the constructed command to be sent to R. This is not a valid R command yet, because of the placeholders `<dataset>` and `<variable>`, but it will be updated as the user makes selections in the dialog. The functions `inside()` and `recode()` are both part of the R package `admisc`.

Hitting the 'Run' button, at this very moment, will trigger a validation error because no dataset or variable has been selected yet:



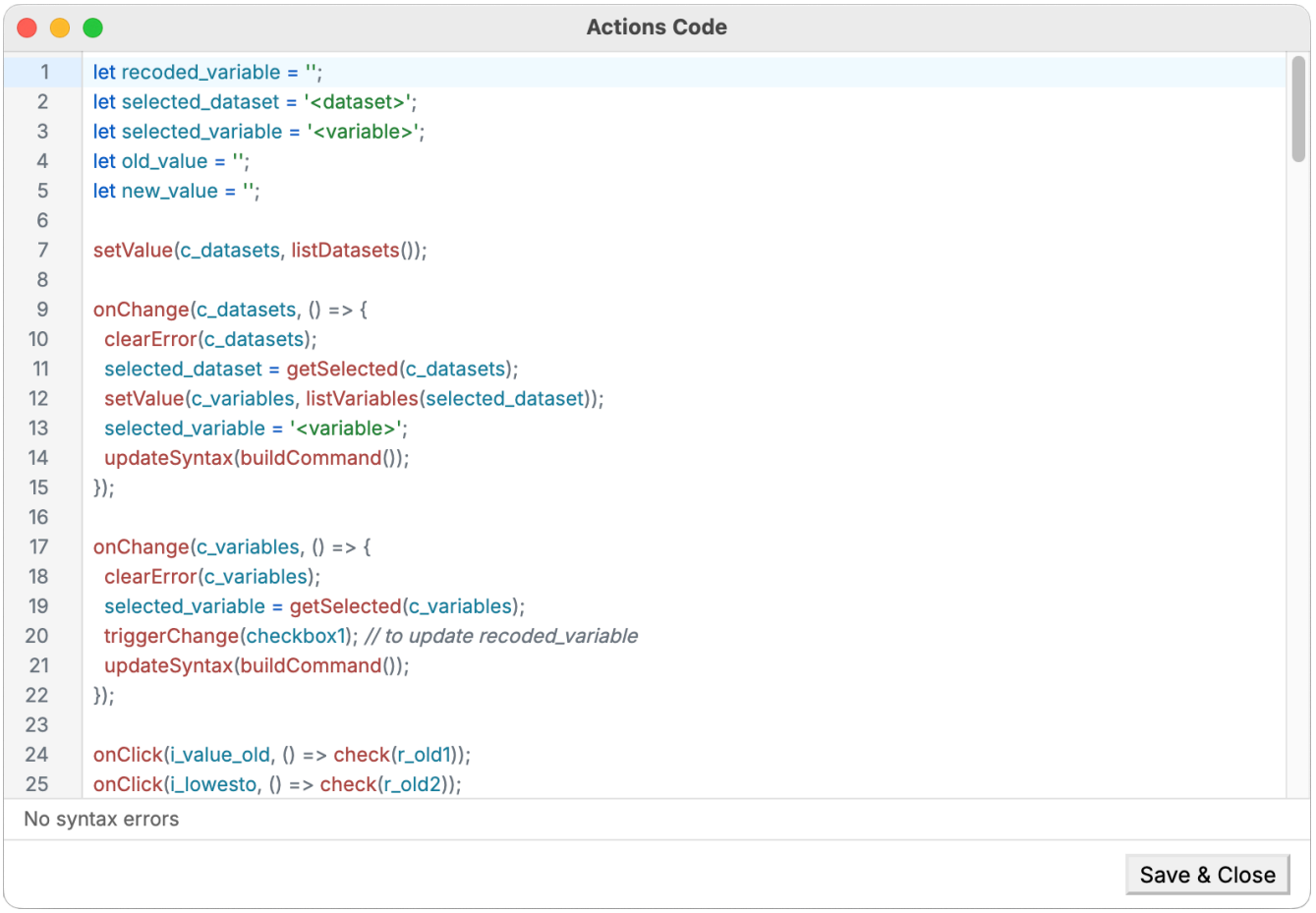
Upon selecting a dataset, in this case 'ToothGrowth', the variables container is populated with the numeric variables from that dataset, namely 'len' and 'dose' (while the categorical variable 'supp' is disabled). The syntax panel is also updated to reflect the selected dataset. Hitting 'Run' now will trigger a different validation error, this time for the missing variable selection:




```
inside(
  ToothGrowth,
  <variable> <- recode(
    <variable>,
    rules = ""
  )
)
```

Note how the syntax panel now shows the selected dataset 'ToothGrowth' instead of the placeholder `<dataset>`, while the variable is still unselected, hence the placeholder `<variable>` remains. This way, the syntax panel is progressively updated as the user makes selections in the dialog. Making use of the Javascript's reactive nature, the syntax panel is updated automatically whenever the user selects or clicks something in the dialog.

This looks like a lot of work, but in reality it only requires a few lines of code to make it all work. Below is an image of the custom scripting area that makes this dialog functional, that appear when the button 'Actions' is clicked in the design window:



The syntax construction is left entirely to the user's imagination, and a dedicated custom function `buildCommand()` will be introduced later. In the above image, the code starts by defining a few global variables to hold the selected dataset and variable names, as well as the recoded variable name (which is updated via a checkbox handler, also shown later).

Once the Preview window is started, the first action is to bind the datasets container to the host application's available data objects. For ordinary dialogs this should be done with `bindObjects()`. The host application then owns the dataset list and any refresh behavior, while the dialog code remains responsible for reacting to the user's selection:

```
bindObjects({
  dialog: 'recode',
  datasets: c_datasets,
  variables: c_variables
});
```

(note also that the container name `c_datasets` is used here, as manually changed in the design window).

The custom code then continues with an event handler for the datasets container, which triggers whenever the user selects a dataset. Inside this handler, the selected dataset is retrieved via `getSelected()`, and stored in the global variable `selected_dataset`. When `variables` is supplied to `bindObjects()`, the host or Preview runtime updates that variable container for the selected dataset. Finally, the syntax panel is updated by calling a custom function `buildCommand()`, which constructs the command string based on the current selections:

```
onChange(c_datasets, () => {
  clearError(c_datasets);
```

```
  selected_dataset = getSelected(c_datasets)[0] || '<dataset>';
  if (selected_dataset === '<dataset>') {
    clearContent(c_variables);
  }
  selected_variable = '<variable>';
  updateSyntax(buildCommand());
});
```

For preview-only scripts or special dialogs that intentionally own their object list, the lower-level manual pattern is still available:

```
setValue(c_datasets, listObjects('datasets'));
```

Do not combine that manual population with `bindObjects()` for the same dataset selector. Pick one owner for the list.

The next set of commands are just convenience handlers for the radio buttons. For instance, when the user clicks on the first top input in the old values (`i_value_old`), the corresponding radion button is checked programmatically via the `check()` API function. Similar handlers are defined for all other radio buttons, both for old and new values:

```
onClick(i_value_old, () => check(r_old1));
onClick(i_lowesto, () => check(r_old2));
onClick(i_from, () => check(r_old3));
onClick(i_to, () => check(r_old3));
onClick(i_tohighest, () => check(r_old4));
onClick(i_value_new, () => check(r_new1));
```

The radio buttons have explicit handlers themselves. Once clicked, they fire the change events for their corresponding input fields, ensuring that the UI stays in sync with the user's selections. This allows for a more dynamic and responsive dialog experience, as changes to one element can automatically update others as needed.

```
onChange(radiogroup1, () => {
  if (isChecked(r_old1)) triggerChange(i_value_old);
  if (isChecked(r_old2)) triggerChange(i_lowesto);
  if (isChecked(r_old3)) triggerChange(i_from); // also checks i_to
  if (isChecked(r_old4)) triggerChange(i_tohighest);
  if (isChecked(r_old5)) old_value = "missing";
  if (isChecked(r_old6)) old_value = "else";
});

onChange(radiogroup2, () => {
  if (isChecked(r_new1)) triggerChange(i_value_new);
  if (isChecked(r_new2)) new_value = 'missing';
  if (isChecked(r_new3)) new_value = 'copy';
});
```

Instead of listening to each individual radio button, the code above listens to the entire radio group `radiogroup1` , and checks which radio button is currently selected. For instance, if the first radio button `r_old1` is checked, it triggers the change event for the corresponding input field `i_value_old` , which will update the `old_value` variable accordingly (and similar logic applies to the corresponding input in the new values section):

```
onChange(i_value_old, () => old_value = getValue(i_value_old));
onChange(i_value_new, () => new_value = getValue(i_value_new));
```

The next input from the old values section is handled similarly, updating the `old_value` variable when the user changes the input:

```
onChange(i_lowesto, () => {
  const lowesto = getValue(i_lowesto);
  old_value = lowesto ? 'lo:' + lowesto : '';
});
```

The part with `old_value = lowesto ? 'lo:' + lowesto : '';` is a Javascript shorthand for:

```
  if (lowesto) {
    old_value = 'lo:' + lowesto;
  } else {
    old_value = '';
  }
```

It is similar to the equivalent R code: `oldvalue <- ifelse(nzchar(lowesto), paste0('lo:', lowesto), '')`

Both next inputs `i_from` and `i_to` from the old values section are handled in a similar manner, updating the `old_value` variable based on user input:

```
// delegate to i_to
onChange(i_from, () => triggerChange(i_to));

onChange(i_to, () => {
  const from = getValue(i_from);
  const to = getValue(i_to);
  old_value = (from && to) ? from + ':' + to : '';
});
```

Here, both "from" and "to" inputs have to be non-empty to construct a valid range string for `old_value`, otherwise it defaults to an empty string. In a similar fashion, the input from the option "to highest" is handled next:

```
onChange(i_tohighest, () => {
  const tohighest = getValue(i_tohighest);
  old_value = tohighest ? tohighest + ':hi' : '';
});
```

If both old and new values have valid content, the 'Add' button (named `b_add` in the editor area) can be clicked to add a new recoding rule into the rules container. The handler for this button first clears any previous validation errors on the rules container, then checks if both `old_value` and `new_value` are non-empty. If either is empty, it adds a descriptive error message to the rules container. Otherwise, it constructs a rule string in the format "old_value = new_value", adds it to the rules container using `addValue()`, clears any previously added errors and finally updates the syntax panel:

```
onClick(b_add, () => {
  if (old_value && new_value) {
    addValue(c_rules, old_value + '=' + new_value);
    clearContent(i_value_old, i_lowesto, i_from, i_to, i_tohighest, i_value_new);
    clearError(c_rules);
    updateSyntax(buildCommand());
  } else if (old_value) {
    addError(c_rules, 'new value not defined');
  } else if (new_value) {
    addError(c_rules, 'old value not defined');
  } else {
    addError(c_rules, 'old and new values needed');
  }
});
```

The next button is 'Remove', which deletes the selected rules from the rules container. Its handler first retrieves the selected rules via `getSelected()`, then removes them one by one using `clearValue()`. Before exiting, it updates the syntax panel:

```
onClick(b_remove, () => {
  clearValue(c_rules, getSelected(c_rules));
  updateSyntax(buildCommand());
});
```

The 'Clear' button removes all rules from the rules container. Its handler simply calls `clearContent()` on the rules container, then updates the syntax panel:

```
onClick(b_clear, () => {
  clearContent(c_rules);
  updateSyntax(buildCommand());
});
```

The checkbox on the bottom left side of the dialog indicates whether the recoding should be done in place or to a new variable. Its handler checks the current state of the checkbox using `isChecked()`, then shows or hides the new variable input accordingly using `show()` and `hide()`. It also updates the global variable `recoded_variable` to either the `selected_variable` (if recoding in place) or to the new variable name `newvar` (if recoding to a new variable, collected from the `i_newvar` input). Finally, it updates the syntax panel:

```
onChange(checkbox1, () => {
  if (isChecked(checkbox1)) {
    show(i_newvar);
    const newvar = getValue(i_newvar);
    recoded_variable = newvar ? newvar : selected_variable;
    updateSyntax(buildCommand());
  } else {
    hide(i_newvar);
    recoded_variable = selected_variable;
    updateSyntax(buildCommand());
  }
});
```

Similar to the previous input handlers, the new variable input has its own change handler that updates the `recoded_variable` variable when changed:

```
onChange(i_newvar, () => {
  clearError(i_newvar);
  const newvar = getValue(i_newvar);
  recoded_variable = newvar ? newvar : selected_variable;
  updateSyntax(buildCommand());
});
```

All of these handlers use the `buildCommand()` function to construct the R command string based on the current selections. This function retrieves all the relevant bits , and builds the final command string in the required format:

```
const buildCommand = () => {
  const rules = getValue(c_rules);
  triggerChange(checkbox1); // to update recoded_variable

  let command = 'inside(\n  ' + selected_dataset + ',\n  ';
  command += recoded_variable + ' <- recode(\n    ';
  command += selected_variable + ',\n    rules = "';
  command += rules ? rules.join('; ') : '';
  command += '"\n  )\n)\n';
  return command;
}
```

Finally, the main 'Run' button validates the user's selections and either shows validation errors or proceeds to execute the constructed command. Its only purpose in the Preview window is to validate the user's input, and add error messages if needed.

```
onClick(b_run, () => {
  if (selected_dataset === '<dataset>') {
    addError(c_datasets, "No dataset selected");
    return;
  }

  if (selected_variable === '<variable>') {
    addError(c_variables, "No variable selected");
    return;
  }

  if (!getValue(c_rules)) {
    addError(c_rules, "No recoding rules");
    return;
  }

  if (isChecked(checkbox1)) {
    const newvar = getValue(i_newvar);
    if (!newvar) {
      addError(i_newvar, "New variable needs a name.")
    }
  } else {
    clearError(i_newvar);
  }

  run(buildCommand());
});
```

At the very end, if all validations pass, the constructed command is sent to R via the `run()` API function. The final command in the code window simply prints the initial constructed syntax when the Preview window is

opened:

```
updateSyntax(buildCommand());
```

This is fired only once, and it can only be placed after the `buildCommand()` function definition, so that the function is already known when called.

Download example

Download the complete recode dialog example: recode.json

This file contains the full dialog configuration used in the case study above, which you can load directly into Dialog Creator to examine the layout, element properties, and actions code.